

IET ASSIGNMENT

DEVELOPMENT AND PERFORMANCE ANALYSIS OF A DECISION TREE-BASED CLASSIFICATION MODEL FOR URBAN WATER QUALITY ASSESSMENT

Student Details	Information
Name	M S Mukilan
Register Number	192412071
Course Code	ITA0609
Course Name	Machine Learning
Slot	A
Assessment Level	L4 – Analyse; L5 – Evaluate; L6 – Create

1. Problem Statement and Objectives

The assignment requires an interpretable Machine Learning solution for classifying urban water samples into suitable quality categories. Historical water-quality records contain physicochemical indicators such as pH, turbidity, dissolved oxygen, electrical conductivity, total dissolved solids, temperature and related measurements. A Decision Tree is selected because its learned structure can be inspected and converted into IF–THEN rules.

Objectives:

- Formulate water-quality classification as a concept-learning problem.
- Prepare and inspect a public water-quality dataset using Pandas.
- Select important attributes using a Decision Tree heuristic such as Gini impurity.
- Construct, visualise and interpret a Decision Tree classifier.
- Classify unseen samples and derive representative IF–THEN decision rules.
- Evaluate performance using Accuracy, Precision, Recall, F1-score and a Confusion Matrix.
- Analyse reliability, overfitting, underfitting, generalisation and practical suitability.
- Discuss the relationship of the solution to SDG 6, SDG 9, SDG 11 and SDG 12.

2. Dataset Selection and Problem Formulation

A publicly available Water Potability dataset is used. It contains 3,276 observations and nine input water-quality attributes. The target variable is Potability, where 1 represents potable water and 0 represents non-potable water.

Attribute	Type	Role
pH	Continuous	Input feature
Hardness	Continuous	Input feature
Solids	Continuous	Input feature
Chloramines	Continuous	Input feature
Sulfate	Continuous	Input feature
Conductivity	Continuous	Input feature
Organic_carbon	Continuous	Input feature
Trihalomethanes	Continuous	Input feature
Turbidity	Continuous	Input feature
Potability	Binary	Target concept

Concept definition:

Target concept $C(x) = 1$ if the water sample is classified as potable and $C(x) = 0$ otherwise. The learning task is to infer C from labelled historical samples.

Training/testing strategy:

The dataset is divided into training and testing subsets. A stratified split is preferred so that the proportion of potable and non-potable samples is approximately preserved in both subsets.

3. Concept Representation, Hypothesis Space and Inductive Bias

Each sample is represented as a feature vector $x = (\text{pH}, \text{Hardness}, \text{Solids}, \text{Chloramines}, \text{Sulfate}, \text{Conductivity}, \text{Organic_carbon}, \text{Trihalomethanes}, \text{Turbidity})$. The hypothesis space consists of decision trees that recursively partition this feature space using threshold tests such as $\text{feature} \leq \text{threshold}$.

Element	Formulation
Instance space X	All valid combinations of the nine water-quality measurements.
Target concept	$\text{Potability} \in \{0, 1\}$.
Hypothesis space H	All decision trees expressible using the available attributes and thresholds.

Inductive bias	Prefer simple, informative splits and a tree that fits training patterns while controlling unnecessary complexity.
Search strategy	Greedy heuristic search: choose the best available split at each node and recurse.

4. Dataset Preparation and Pandas Analysis

Required preparation operations include:

1. Load the CSV file with `pandas.read_csv()`.
2. Inspect dimensions, column names, data types and summary statistics.
3. Check missing values and duplicate records.
4. Handle missing numeric values using a documented strategy such as median imputation.
5. Separate X (features) and y (Potability).
6. Use filtering, grouping and sorting to inspect quality-class patterns.
7. Split the data into training and testing sets using stratification.

Representative Pandas operations:

import pandas as pd

```
df = pd.read_csv("water_potability.csv")
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
print(df["Potability"].value_counts())
```

```
# Example filtering, grouping and sorting
print(df[df["pH"] < 6.5].head())
print(df.groupby("Potability")["pH"].mean())
print(df.sort_values("Turbidity", ascending=False).head())
```

5. Heuristic Attribute Selection using Gini Impurity

Gini impurity is selected because it is simple, computationally efficient and directly supported by the `DecisionTreeClassifier` implementation. For a node containing class proportions $p_1, p_2, \dots, p_k$:

$$\text{Gini}(S) = 1 - \sum p_i^2$$

For a candidate split, the weighted impurity is calculated as:

$$\text{Gini_split} = (|S_1|/|S|)\text{Gini}(S_1) + (|S_2|/|S|)\text{Gini}(S_2)$$

The best split is the one producing the lowest weighted Gini impurity (equivalently, the largest impurity reduction). The algorithm repeats this greedy selection recursively until a stopping condition is met.

Illustrative calculation:

Suppose a node contains 100 samples: 60 non-potable and 40 potable. Then $\text{Gini} = 1 - (0.6^2 + 0.4^2) = 0.48$. If a candidate split creates child nodes of 50/10 and 10/30 for the two classes, their Gini values are 0.278 and 0.375 respectively. The weighted split impurity is $(60/100)(0.278) + (40/100)(0.375) = 0.3168$. The impurity reduction is $0.48 - 0.3168 = 0.1632$. The candidate with the greatest reduction is preferred.

6. Decision Tree Learning: Pseudocode and Construction

Pseudocode:

```
DECISION_TREE_LEARN(dataset, features):
    if all samples have the same class:
        return leaf(class)

    if features are empty or stopping condition is reached:
        return leaf(majority_class(dataset))

    best_split = split with minimum weighted Gini impurity

    create a decision node using best_split

    for each child partition:
        child_tree = DECISION_TREE_LEARN(child_data, remaining_features)
        attach child_tree to the decision node

    return decision node
```

Important stopping conditions:

- All samples at the node belong to one class.
- Maximum tree depth is reached.
- Minimum number of samples required for a split is not satisfied.
- No useful impurity reduction remains.

7. Python Implementation and Menu-Driven Functionality

The following implementation uses Pandas for data manipulation and scikit-learn for the Decision Tree, train/test split and evaluation. The menu allows the user to inspect the dataset, train the model, display metrics and predict a new sample.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import matplotlib.pyplot as plt

df = pd.read_csv("water_potability.csv")

# Preprocessing
df = df.drop_duplicates()
df[df.columns] = df[df.columns].apply(pd.to_numeric, errors="coerce")
df = df.fillna(df.median(numeric_only=True))

X = df.drop("Potability", axis=1)
y = df["Potability"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    min_samples_split=10,
    random_state=42
```

```

)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall :", recall_score(y_test, y_pred, zero_division=0))
print("F1-score :", f1_score(y_test, y_pred, zero_division=0))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Tree visualisation
plt.figure(figsize=(18, 10))
plot_tree(model, feature_names=X.columns, class_names=["Non-potable", "Potable"],
          filled=True, rounded=True)
plt.title("Decision Tree for Water Potability")
plt.show()

# Feature importance
importance = pd.Series(model.feature_importances_, index=X.columns)
print(importance.sort_values(ascending=False))

```

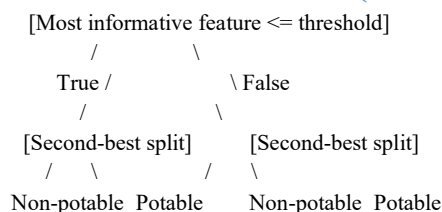
Menu structure:

1. Display dataset information
2. Show summary statistics
3. Check missing values
4. Train Decision Tree
5. Display evaluation metrics
6. Display feature importance
7. Visualise Decision Tree
8. Predict a new water sample
9. Exit

8. Decision Tree Visualisation and Analysis

The learned tree should be visualised using `plot_tree()`. Each internal node contains a feature-threshold test, impurity and sample/class information. Leaf nodes show the predicted class.

Representative tree structure (conceptual):



The exact features, thresholds and branches are produced by the trained dataset/model.

Feature importance values can be inspected from `model.feature_importances_`. A larger value indicates that the corresponding attribute contributed more to impurity reduction across the tree. The tree should not be interpreted as proving that a feature causes poor water quality; it only indicates predictive usefulness.

9. Classification and IF–THEN Decision Rules

A new sample is passed through the tree from the root. At each node, the relevant measurement is compared with the learned threshold. The process continues until a leaf is reached.

Representative rule format:

```
IF Feature_A <= threshold_1
  AND Feature_B > threshold_2
THEN Potability = 1 (Potable)
```

```
IF Feature_A > threshold_1
  AND Feature_C <= threshold_3
THEN Potability = 0 (Non-potable)
```

The exact rules must be extracted from the actual trained tree because thresholds depend on the selected training data and model parameters.

10. Performance Evaluation

The model is evaluated on unseen test samples using the following measures:

Metric	Formula / Meaning
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$
Precision	$TP / (TP + FP)$; reliability of positive predictions
Recall	$TP / (TP + FN)$; ability to identify potable samples
F1-score	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$
Confusion Matrix	Shows TN, FP, FN and TP prediction counts.

For water-quality screening, accuracy alone is insufficient. Precision and recall should both be examined because different errors have different practical consequences. The confusion matrix makes these error types explicit.

Results table to be generated by the program:

Measure	Test-set result
Accuracy	Generated when the program is executed
Precision	Generated when the program is executed
Recall	Generated when the program is executed
F1-score	Generated when the program is executed
Confusion Matrix	Generated when the program is executed

11. Reliability, Overfitting, Underfitting and Generalisation

Overfitting

A very deep tree can memorise training samples, producing high training performance but weaker test performance. Limiting max_depth and increasing min_samples_split can reduce this risk.

Underfitting

An excessively shallow tree may fail to capture useful relationships. This can reduce both training and test performance.

Generalisation

The final model should be judged primarily on unseen test data. Stratification and a fixed random state make the experiment reproducible.

Reliability

A reliable model should be interpretable, reproducible and reasonably stable under small data changes. Cross-validation can be added for stronger evidence.

Limitations

The dataset may not represent every geographic location, season or contamination source. A binary potability label also simplifies the wider problem of water-quality assessment.

12. CO3 Analysis: Neural Network Concepts

Although the implemented model is a Decision Tree, the assignment rubric also assesses understanding of Perceptron, Multilayer Perceptron (MLP) and Back Propagation.

Concept	Analysis
Perceptron	A single-layer linear classifier. It is simple but cannot represent non-linearly separable relationships.
MLP	Uses one or more hidden layers and nonlinear activation functions to learn more complex patterns.
Back Propagation	Computes the error gradient through the network and updates weights to reduce the loss.
Comparison with Decision Tree	Decision Trees are generally easier to interpret as rules; neural networks can model complex nonlinear relationships but are less transparent.

13. CO4 Analysis: Genetic Algorithm and Hypothesis-Space Search

A Genetic Algorithm (GA) searches a solution space using a population of candidate solutions. A decision-tree structure or hyperparameter configuration can be represented as a chromosome. Fitness can combine predictive performance with a complexity penalty.

8. Initialise a population of candidate tree structures/parameters.
9. Evaluate each candidate using validation performance.
10. Select better candidates according to fitness.
11. Apply crossover and mutation to generate new candidates.
12. Repeat until a stopping criterion is reached.
13. Choose the best validated solution.

Compared with greedy Decision Tree learning, GA search can explore a broader hypothesis space but usually requires more computation and careful fitness design.

14. SDG Relevance

SDG	Relevance to the project
SDG 6 – Clean Water and Sanitation	Automated water-quality classification can support faster identification of samples requiring investigation or treatment.
SDG 9 – Industry, Innovation and Infrastructure	Machine Learning provides a data-driven approach that can be integrated into monitoring infrastructure.
SDG 11 – Sustainable Cities and Communities	Reliable water monitoring contributes to safer and more resilient urban services.
SDG 12 – Responsible Consumption and Production	Early identification of water-quality problems can support efficient treatment and responsible resource management.

15. Reflection, Design Decisions and Learning Outcomes

The main design decision was to use a Decision Tree because the assignment emphasises interpretability and heuristic attribute selection. Gini impurity provides a clear mathematical basis for selecting splits. Pandas supports transparent inspection and manipulation of the dataset, while a held-out test set provides evidence of generalisation.

A key challenge is avoiding unsupported conclusions from a single dataset. The model should therefore be treated as a screening/decision-support tool rather than a replacement for laboratory testing. Another challenge is balancing tree complexity against predictive performance.

Learning outcomes achieved:

- Formulated a real-world classification task as concept learning.
- Defined attributes, target concept, hypothesis space and inductive bias.
- Applied Gini impurity as a heuristic search criterion.
- Used Pandas for inspection, filtering, grouping and sorting.
- Implemented and visualised a Decision Tree.
- Extracted interpretable IF-THEN decision rules.
- Evaluated classification performance and discussed generalisation.
- Connected the technical solution to relevant Sustainable Development Goals.

16. Conclusion and Recommendation

The Decision Tree approach provides an interpretable framework for urban water-quality classification. Its major advantage is the ability to convert learned splits into understandable rules, which is valuable for operational screening. Performance should be confirmed on the held-out test set and, preferably, through cross-validation and external datasets before deployment.

Recommendation: use the Decision Tree as an interpretable screening and decision-support component, while retaining laboratory analysis and domain-expert review for critical water-safety decisions.

17. GitHub Upload Deliverable

The assignment brief specifies GitHub upload as a deliverable. The repository should contain at minimum:

- README.md with project description, dataset source, installation and execution steps.
- water_potability.csv or a clear dataset download instruction, subject to dataset licensing.
- decision_tree_water_quality.py
- requirements.txt
- Decision Tree visualisation image.
- Sample output/metrics.
- This assignment report.

Suggested README execution commands:

```
pip install pandas scikit-learn matplotlib  
python decision_tree_water_quality.py
```

18. References

14. Assignment brief: IET ASSIGNMENT SLOT A – Machine Learning, provided course document.
15. Breiman, L., Friedman, J., Olshen, R. A., & Stone, C. J. (1984). Classification and Regression Trees. Wadsworth.
16. Quinlan, J. R. (1993). C4.5: Programs for Machine Learning. Morgan Kaufmann.
17. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

18. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

Appendix A – Complete Reproducible Program

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

FILE = "water_potability.csv"

def load_prepare():
    df = pd.read_csv(FILE)
    df = df.drop_duplicates()
    df[df.columns] = df[df.columns].apply(pd.to_numeric, errors="coerce")
    df = df.fillna(df.median(numeric_only=True))
    return df

def train_model(df):
    X = df.drop("Potability", axis=1)
    y = df["Potability"]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )
    model = DecisionTreeClassifier(
        criterion="gini",
        max_depth=5,
        min_samples_split=10,
        random_state=42
    )
    model.fit(X_train, y_train)
    return model, X_train, X_test, y_train, y_test

def evaluate(model, X_test, y_test):
    pred = model.predict(X_test)
    print("\nAccuracy :", accuracy_score(y_test, pred))
    print("Precision:", precision_score(y_test, pred, zero_division=0))
    print("Recall   :", recall_score(y_test, pred, zero_division=0))
    print("F1-score :", f1_score(y_test, pred, zero_division=0))
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, pred))
    print("\nClassification Report:\n", classification_report(
        y_test, pred, zero_division=0
    ))

def visualise(model, feature_names):
    plt.figure(figsize=(18, 10))
    plot_tree(
        model,
        feature_names=feature_names,
        class_names=["Non-potable", "Potable"],
        filled=True,
        rounded=True
    )
    plt.title("Decision Tree for Water Potability")
    plt.tight_layout()
    plt.show()

def main():
    df = load_prepare()
```

```

model, X_train, X_test, y_train, y_test = train_model(df)

while True:
    print("\n--- WATER QUALITY DECISION TREE ---")
    print("1. Dataset information")
    print("2. Summary statistics")
    print("3. Missing values")
    print("4. Evaluate model")
    print("5. Feature importance")
    print("6. Visualise tree")
    print("7. Predict new sample")
    print("8. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        print(df.info())
    elif choice == "2":
        print(df.describe())
    elif choice == "3":
        print(df.isnull().sum())
    elif choice == "4":
        evaluate(model, X_test, y_test)
    elif choice == "5":
        imp = pd.Series(model.feature_importances_, index=X_train.columns)
        print(imp.sort_values(ascending=False))
    elif choice == "6":
        visualise(model, X_train.columns)
    elif choice == "7":
        print("Enter values in this order:")
        print(list(X_train.columns))
        values = [float(v) for v in input().split()]
        sample = pd.DataFrame([values], columns=X_train.columns)
        print("Predicted class:", model.predict(sample)[0])
    elif choice == "8":
        break
    else:
        print("Invalid choice.")

if __name__ == "__main__":
    main()

```